



1. Problem Title & Link

Title: LeetCode 763 : Partition Labels

Link: [LeetCode 763 - Partition Labels](#)

2. Problem Statement (Short Summary)

You are given a string:

s

Partition the string into as many parts as possible such that:

each letter appears in at most one partition

Return a list containing the size of each partition.

In simple terms:

Split the string into sections where no character appears in more than one section.

3. Examples (Input → Output)

Example 1

Input:

s="ababcbacadefegdehijhklij"

Output:

[9,7,8]

Explanation:

Partitions:

"ababcbaca"

"defegde"

"hijhklij"

Characters do not repeat across partitions.

Example 2

Input:

s="eccbbbbbdec"

Output:

[10]

4. Constraints

1 <= s.length <=500

s contains lowercase English letters



Special restrictions:

- Maximize number of partitions
- Characters should belong to only one partition

5. Core Concept (Pattern / Topic)

Greedy + HashMap

6. Thought Process (Step-by-Step Explanation)

Brute Force

Try every possible split:

1. Create partitions
2. Verify character overlap
3. Keep maximum valid partitions

Time:

$O(n^2)$

Too slow.

Optimized Thinking

Observation:

For every character:

find its last occurrence

While traversing:

- Maintain current partition end
- Update:

end=max(

end,

lastOccurrence[currentChar]

)

If current index becomes:

$i == \text{end}$

then:

partition ends here

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

ababcbacadefegdehijhklij

Last occurrences:



$a \rightarrow 8$
 $b \rightarrow 5$
 $c \rightarrow 7$
 $d \rightarrow 14$
 $e \rightarrow 15$
 ...
 Traversal:
 a b a b c b a c a

↑

end=8

Reached:

i=8

Partition:

length=9

Continue similarly.

8. Pseudocode (Language Independent)

store last occurrence

start=0

end=0

result=[]

for i from 0 to n-1:

end=

max(

end,

last[s[i]]

)

if i==end:

result.add(



```
        end-start+1
    )
```

```
    start=i+1
```

```
return result
```

9. Code Implementation

Python

class Solution:

```
    def partitionLabels(
        self,
        s):
```

```
        last={}

```

```
        for i in range(
            len(s)
        ):
```

```
            last[s[i]]=i

```

```
        result=[]

```

```
        start=0
        end=0
```

```
        for i in range(
            len(s)
        ):
```

```
            end=max(
                end,
                last[s[i]]
            )
```



```
        if i==end:

            result.append(
                end-start+1
            )

            start=i+1

    return result
```

Java

```
class Solution {

    public List<Integer>
    partitionLabels(
        String s) {

        int[] last=
            new int[26];

        for(int i=0;
            i<s.length();
            i++){

            last[
                s.charAt(i)
                -'a'
            ]=i;
        }

        List<Integer>
        result=
            new ArrayList<>();

        int start=0;
        int end=0;
```



```
for(int i=0;
    i<s.length();
    i++){

    end=
        Math.max(
            end,
            last[
                s.charAt(i)
                -'a'
            ]
        );

    if(i==end){

        result.add(
            end-start+1
        );

        start=i+1;
    }
}

return result;
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

- One pass to store last occurrences
- One pass to create partitions

Space Complexity:

$O(1)$



Reason:

Maximum:

26 characters

stored.

11. Common Mistakes / Edge Cases

1. Using first occurrence instead of last occurrence

Wrong:

$a \rightarrow 0$

Need:

$a \rightarrow 8$

2. Forgetting:

$start=i+1$

after partition

3. Missing single-character case

$s="a"$

Output:

[1]

4. Using nested loops

12. Detailed Dry Run (Step-by-Step Table)

Input:

$s="ababcbacadeffegdehijklij"$

Index	Character	End	Partition
0	a	8	-
1	b	8	-
2	a	8	-
...	...	...	-
8	a	8	Length=9
15	e	15	Length=7
23	j	23	Length=8

Output:



[9,7,8]

13. Common Use Cases (Real-Life / Interview)

Log segmentation

Task grouping systems

Dependency isolation

Resource partitioning

14. Common Traps (Important!)

1. Using first occurrence
2. Incorrect partition length formula
3. Forgetting partition reset
4. Using brute force

15. Builds To (Related LeetCode Problems)

Progression:

- LC 763 → Partition Labels
- LC 56 → Merge Intervals
- LC 435 → Non-overlapping Intervals
- LC 621 → Task Scheduler
- LC 452 → Minimum Number of Arrows to Burst Balloons

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(n)$	Repeated checking
HashMap + Greedy	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

Each partition must extend until the last occurrence of every character inside it. By continuously updating the partition boundary, we guarantee that no character appears in future partitions.

18. Variations / Follow-Up Questions

1. Return actual partition strings
2. Partition by words instead of characters
3. Minimize partition count



4. Support Unicode characters
5. Dynamic partition updates
- 6.